

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
АДЫГЕЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
НОК “Институт точных наук и цифровых технологий”
Кафедра автоматизированных систем обработки информации и управления

ОТЧЕТ ПО ПРАКТИКЕ

Лабораторная работа №2

Решение системы линейных алгебраических уравнений методом
простой итерации

Вариант №1

Точность вычислений: $\varepsilon = 0.01$

1 курс, группа 2ИВТ

Выполнил:

_____ Д. А. Скрылев

Руководитель:

_____ С. В. Теплоухов

Майкоп, 2026 г.

Содержание

1	Постановка задачи	2
2	Теоретические сведения	2
3	Проверка сходимости	2
4	Переход к итерационному виду	3
5	Программа	3
5.1	Основной фрагмент исходного кода	3
6	Таблица итераций	5
7	Скриншот запуска программы	6
8	Результаты	7
9	Физическая задача 2.1	7
9.1	Постановка задачи	7
9.2	Закон сохранения заряда	8
9.3	Расчёт до замыкания ключа	8
9.4	Узловые уравнения после замыкания ключа	8
9.5	Энергия после замыкания	9
9.6	Изменение энергии	9
9.7	Программа	9
9.8	Скриншот запуска программы	11
9.9	Вывод по физической задаче	11
10	Вывод	12

1 Постановка задачи

Необходимо решить систему линейных алгебраических уравнений

$$AX = B$$

с точностью $\varepsilon = 0.01$. По варианту №1 используется метод простой итерации.

Исходные данные:

$$A = \begin{pmatrix} 5 & 0 & 1 \\ 1 & 3 & -1 \\ -3 & 2 & 10 \end{pmatrix}, \quad B = \begin{pmatrix} 11 \\ 4 \\ 6 \end{pmatrix}.$$

Контрольный ответ:

$$X \approx \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}.$$

2 Теоретические сведения

Система линейных алгебраических уравнений $AX = B$ состоит из матрицы коэффициентов A , вектора неизвестных X и вектора свободных членов B . Решением системы является такой вектор X , при подстановке которого все уравнения системы выполняются.

Метод простой итерации относится к численным методам. Система приводится к виду

$$X^{(n+1)} = CX^{(n)} + d,$$

где C — матрица итераций, d — вектор свободных членов итерационного процесса, n — номер итерации.

Начальное приближение в работе выбрано равным

$$X^{(0)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

Если норма матрицы итераций $q = \|C\|_\infty$ меньше единицы, то итерационный процесс сходится. Для контроля точности используется апостериорная оценка:

$$R_n = \frac{q}{1-q} \|X^{(n)} - X^{(n-1)}\|_\infty.$$

Вычисления прекращаются при выполнении условия

$$R_n \leq \varepsilon.$$

3 Проверка сходимости

Для данной системы выполняется строгое диагональное преобладание по строкам:

$$|5| > |0| + |1|, \quad |3| > |1| + |-1|, \quad |10| > |-3| + |2|.$$

Следовательно, система подходит для применения метода простой итерации, а итерационный процесс должен сходиться.

4 Переход к итерационному виду

Исходная система имеет вид:

$$\begin{cases} 5x_1 + x_3 = 11, \\ x_1 + 3x_2 - x_3 = 4, \\ -3x_1 + 2x_2 + 10x_3 = 6. \end{cases}$$

Выразим каждую переменную через остальные:

$$\begin{cases} x_1 = \frac{11 - x_3}{5}, \\ x_2 = \frac{4 - x_1 + x_3}{3}, \\ x_3 = \frac{6 + 3x_1 - 2x_2}{10}. \end{cases}$$

Итерационные формулы:

$$\begin{cases} x_1^{(n+1)} = \frac{11 - x_3^{(n)}}{5}, \\ x_2^{(n+1)} = \frac{4 - x_1^{(n)} + x_3^{(n)}}{3}, \\ x_3^{(n+1)} = \frac{6 + 3x_1^{(n)} - 2x_2^{(n)}}{10}. \end{cases}$$

В матричном виде:

$$C = \begin{pmatrix} 0 & 0 & -0.2 \\ -\frac{1}{3} & 0 & \frac{1}{3} \\ 0.3 & -0.2 & 0 \end{pmatrix}, \quad d = \begin{pmatrix} 2.2 \\ \frac{4}{3} \\ 0.6 \end{pmatrix}.$$

Норма матрицы итераций:

$$\|C\|_{\infty} = \max(0.2, 0.666667, 0.5) = 0.666667 < 1.$$

Это подтверждает сходимость метода.

5 Программа

Программа написана на языке Python. Она хранит матрицу A и вектор B , проверяет диагональное преобладание, строит итерационную матрицу C , выполняет итерации до достижения заданной точности, выводит таблицу итераций и проверяет невязку $AX - B$.

5.1 Основной фрагмент исходного кода

Полный исходный код программы находится в файле `main.py`. Ниже приведен основной фрагмент, отвечающий за построение итерационной системы и выполнение метода простой итерации.

```
EPSILON = 0.01
A = (
    (5.0, 0.0, 1.0),
```

```

    (1.0, 3.0, -1.0),
    (-3.0, 2.0, 10.0),
)
B = (11.0, 4.0, 6.0)
X0 = (0.0, 0.0, 0.0)

def build_iteration_system(matrix, vector):
    c_rows = []
    d_values = []

    for i, row in enumerate(matrix):
        diagonal = row[i]
        c_row = []

        for j, value in enumerate(row):
            c_row.append(0.0 if i == j else -value / diagonal)

        c_rows.append(tuple(c_row))
        d_values.append(vector[i] / diagonal)

    return tuple(c_rows), tuple(d_values)

def matrix_vector_product(matrix, vector):
    return tuple(
        sum(value * vector[j] for j, value in enumerate(row))
        for row in matrix
    )

def simple_iteration(matrix, vector, initial, epsilon):
    c_matrix, d_vector = build_iteration_system(matrix, vector)
    q = max(sum(abs(value) for value in row) for row in c_matrix)
    previous = initial
    rows = []

    for n in range(1, 101):
        product = matrix_vector_product(c_matrix, previous)
        current = tuple(product[i] + d_vector[i] for i in range(len(product)))
        delta = max(abs(current[i] - previous[i]) for i in range(len(current)))
        error_estimate = q / (1.0 - q) * delta
        rows.append((n, current, delta, error_estimate))

        if error_estimate <= epsilon:
            return current, rows

        previous = current

    raise RuntimeError("Simple iteration did not converge.")

```

6 Таблица итераций

В таблице обозначено:

$$\Delta_n = \|X^{(n)} - X^{(n-1)}\|_\infty, \quad R_n = \frac{q}{1-q} \Delta_n.$$

Таблица 1: Таблица итераций метода простой итерации

n	x_1	x_2	x_3	Δ_n	R_n
1	2.200000	1.333333	0.600000	2.200000	4.400000
2	2.080000	0.800000	0.993333	0.533333	1.066667
3	2.001333	0.971111	1.064000	0.171111	0.342222
4	1.987200	1.020889	1.006178	0.057822	0.115644
5	1.998764	1.006326	0.991982	0.014563	0.029126
6	2.001604	0.997739	0.998364	0.008587	0.017173
7	2.000327	0.998920	1.000933	0.002569	0.005138

На последней итерации $R_n \leq 0.01$, поэтому требуемая точность достигнута.

7 Скриншот запуска программы

```
python3 main.py
```

Лабораторная работа 2: решение СЛАУ

Вариант: 1

Метод: метод простой итерации

Точность: $\epsilon = 0.01$

Начальное приближение: $X_0 = [0.000000, 0.000000, 0.000000]$

Система: $A * X = B$

Матрица A:

```
[ 5.000000,  0.000000,  1.000000]
[ 1.000000,  3.000000, -1.000000]
[-3.000000,  2.000000, 10.000000]
```

Вектор B:

```
[11.000000, 4.000000, 6.000000]
```

Проверка диагонального преобладания:

строка 1: $|a_{11}| = 5.000000 > 1.000000 \rightarrow$ да

строка 2: $|a_{22}| = 3.000000 > 2.000000 \rightarrow$ да

строка 3: $|a_{33}| = 10.000000 > 5.000000 \rightarrow$ да

Итерационный вид: $X(n+1) = C * X(n) + d$

Матрица C:

```
[ 0.000000,  0.000000, -0.200000]
[-0.333333,  0.000000,  0.333333]
[ 0.300000, -0.200000,  0.000000]
```

Вектор d:

```
[2.200000, 1.333333, 0.600000]
```

Норма $\|C\|_{\infty} = q = 0.666667$

Критерий остановки: $q / (1 - q) * \max \text{diff} \leq 0.010000$

Таблица итераций:

n	x1	x2	x3	max diff	err est	max AX-B
1	2.200000	1.333333	0.600000	2.200000	4.400000	3.933333
2	2.080000	0.800000	0.993333	0.533333	1.066667	0.706667
3	2.001333	0.971111	1.064000	0.171111	0.342222	0.578222
4	1.987200	1.020889	1.006178	0.057822	0.115644	0.141956
5	1.998764	1.006326	0.991982	0.014563	0.029126	0.063819
6	2.001604	0.997739	0.998364	0.008587	0.017173	0.025691
7	2.000327	0.998920	1.000933	0.002569	0.005138	0.006191

Финальный ответ:

$X = [2.000327, 0.998920, 1.000933]$

После округления до 0.01: $X = [2.00, 1.00, 1.00]$

Невязка $AX - B = [0.002569, -0.003845, 0.006191]$

$\|AX - B\|_{\infty} = 0.006191$

Рис. 1: Результат запуска программы

8 Результаты

После выполнения итераций получен вектор:

$$X \approx \begin{pmatrix} 2.000327 \\ 0.998920 \\ 1.000933 \end{pmatrix}.$$

После округления до сотых:

$$X \approx \begin{pmatrix} 2.00 \\ 1.00 \\ 1.00 \end{pmatrix}.$$

Невязка для найденного приближенного решения:

$$AX - B \approx \begin{pmatrix} 0.002569 \\ -0.003845 \\ 0.006191 \end{pmatrix}, \quad \|AX - B\|_{\infty} \approx 0.006191.$$

Контрольная проверка для точного округленного ответа:

$$A \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 11 \\ 4 \\ 6 \end{pmatrix} = B.$$

9 Физическая задача 2.1

9.1 Постановка задачи

Батарея конденсаторов (рис. 2) заряжена до разности потенциалов

$$U_0 = 200 \text{ В}.$$

После этого батарея отключена от источника напряжения. Необходимо определить, как изменится энергия батареи при замыкании ключа K .

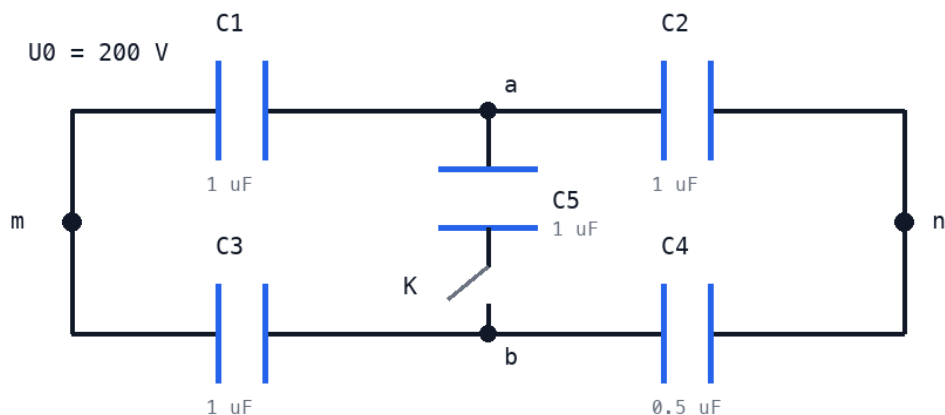


Рис. 2: Схема батареи конденсаторов

Исходные данные:

$$C_1 = C_2 = C_3 = C_5 = 1 \text{ мкФ}, \quad C_4 = 0.5 \text{ мкФ}.$$

При разомкнутом ключе K конденсатор C_5 отключен от нижнего узла и не участвует в эквивалентной ёмкости батареи. После замыкания ключа C_5 подключается между внутренними узлами a и b .

9.2 Закон сохранения заряда

До отключения от источника батарея накапливает заряд $Q = C_{\text{нач}} \cdot U_0$. После отключения внешние клеммы остаются изолированными, поэтому общий заряд батареи сохраняется. Это означает, что при замыкании ключа перераспределение зарядов между конденсаторами происходит при неизменном суммарном заряде батареи, но эквивалентная ёмкость и напряжение меняются.

9.3 Расчёт до замыкания ключа

При разомкнутом ключе C_5 не участвует в цепи. Остаются две параллельные ветви:

- верхняя: C_1 и C_2 последовательно;
- нижняя: C_3 и C_4 последовательно.

Последовательное соединение: $C_{\text{посл}} = \frac{C_a C_b}{C_a + C_b}$.

$$C_{12} = \frac{C_1 C_2}{C_1 + C_2} = \frac{1 \cdot 1}{1 + 1} = 0.5 \text{ мкФ},$$

$$C_{34} = \frac{C_3 C_4}{C_3 + C_4} = \frac{1 \cdot 0.5}{1 + 0.5} = \frac{1}{3} \text{ мкФ},$$

$$C_{\text{нач}} = C_{12} + C_{34} = 0.5 + \frac{1}{3} = \frac{5}{6} \approx 0.8333 \text{ мкФ}.$$

Начальный заряд:

$$Q_{\text{нач}} = C_{\text{нач}} \cdot U_0 = \frac{5}{6} \cdot 200 \approx 166.667 \text{ мкКл}.$$

Начальная энергия:

$$E_{\text{нач}} = \frac{C_{\text{нач}} \cdot U_0^2}{2} = \frac{\frac{5}{6} \cdot 10^{-6} \cdot (200)^2}{2} \approx 0.016667 \text{ Дж}.$$

9.4 Узловые уравнения после замыкания ключа

После замыкания ключа C_5 соединяет узлы a и b . Для нахождения эквивалентной ёмкости $C_{\text{кон}}$ зададим пробное напряжение $V_m = 1 \text{ В}$ и $V_n = 0 \text{ В}$ между внешними клеммами.

Для изолированных внутренних узлов сумма входящих зарядов равна нулю.

Узел a :

$$C_1(V_a - V_m) + C_2(V_a - V_n) + C_5(V_a - V_b) = 0.$$

Узел b :

$$C_3(V_b - V_m) + C_4(V_b - V_n) + C_5(V_b - V_a) = 0.$$

В матричном виде:

$$\begin{cases} (C_1 + C_2 + C_5)V_a - C_5V_b = C_1V_m + C_2V_n, \\ -C_5V_a + (C_3 + C_4 + C_5)V_b = C_3V_m + C_4V_n. \end{cases}$$

Подстановка численных значений:

$$\begin{pmatrix} 3 & -1 \\ -1 & 2.5 \end{pmatrix} \begin{pmatrix} V_a \\ V_b \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Решение:

$$V_a = \frac{7}{13} \approx 0.5385 \text{ В}, \quad V_b = \frac{8}{13} \approx 0.6154 \text{ В}.$$

Общий заряд, пришедший с левой внешней клеммы при пробном напряжении 1 В:

$$Q_{\text{лев}} = C_1(V_m - V_a) + C_3(V_m - V_b) = 1 \cdot \left(1 - \frac{7}{13}\right) + 1 \cdot \left(1 - \frac{8}{13}\right) = \frac{6}{13} + \frac{5}{13} = \frac{11}{13} \text{ мкКл}.$$

Эквивалентная ёмкость после замыкания:

$$C_{\text{кон}} = \frac{Q_{\text{лев}}}{1 \text{ В}} = \frac{11}{13} \approx 0.8462 \text{ мкФ}.$$

9.5 Энергия после замыкания

После отключения источника заряд сохраняется:

$$Q_{\text{кон}} = Q_{\text{нач}} \approx 166.667 \text{ мкКл}.$$

Новое напряжение:

$$U_{\text{кон}} = \frac{Q_{\text{нач}}}{C_{\text{кон}}} \approx \frac{166.667}{0.8462} \approx 196.97 \text{ В}.$$

Конечная энергия:

$$E_{\text{кон}} = \frac{Q_{\text{нач}}^2}{2C_{\text{кон}}} = \frac{(166.667 \cdot 10^{-6})^2}{2 \cdot 0.8462 \cdot 10^{-6}} \approx 0.016414 \text{ Дж}.$$

9.6 Изменение энергии

$$\Delta E = E_{\text{кон}} - E_{\text{нач}} \approx 0.016414 - 0.016667 = -0.000253 \text{ Дж} = -0.253 \text{ мДж}.$$

Энергия батареи уменьшилась примерно на 0.253 мДж. Потерянная энергия переходит в тепло, излучение и искровые разряды при замыкании ключа.

9.7 Программа

Для автоматизации вычислений написан скрипт на языке Python. Он вычисляет $C_{\text{нач}}$, $Q_{\text{нач}}$, $E_{\text{нач}}$, решает узловую систему 2×2 для V_a и V_b , находит $C_{\text{кон}}$, $U_{\text{кон}}$, $E_{\text{кон}}$ и ΔE . Также скрипт записывает текстовый вывод и структурированные результаты в каталог `physics-2.1/artifacts`.

Полный исходный код находится в файле `physics-2.1/main.py`. Ниже приведен основной фрагмент.

```
def compute_initial_state() -> CapacitorBatteryState:
    c12 = capacitors_series(C1, C2)
    c34 = capacitors_series(C3, C4)
    c_initial = c12 + c34
    q_initial = c_initial * U0
    e_initial = energy_from_capacitance_and_voltage(c_initial, U0)
```

```

    return CapacitorBatteryState(
        c_equiv_uf=c_initial,
        q_total_uc=q_initial,
        voltage_v=U0,
        energy_j=e_initial,
    )

def solve_node_potentials() -> NodePotentialSolution:
    vm, vn = 1.0, 0.0

    a11 = C1 + C2 + C5
    a12 = -C5
    b1 = C1 * vm + C2 * vn
    a21 = -C5
    a22 = C3 + C4 + C5
    b2 = C3 * vm + C4 * vn

    det = a11 * a22 - a12 * a21
    va = (b1 * a22 - a12 * b2) / det
    vb = (a11 * b2 - b1 * a21) / det

    q_left = C1 * (vm - va) + C3 * (vm - vb)
    c_final = q_left / vm

    return NodePotentialSolution(
        va_v=va,
        vb_v=vb,
        q_left_uc=q_left,
        c_equiv_uf=c_final,
    )

def solve_final_state(q_initial_uc: float, c_final_uf: float) ->
    CapacitorBatteryState:
    u_final = q_initial_uc / c_final_uf
    e_final = energy_from_charge_and_capacitance(q_initial_uc, c_final_uf)

    return CapacitorBatteryState(
        c_equiv_uf=c_final_uf,
        q_total_uc=q_initial_uc,
        voltage_v=u_final,
        energy_j=e_final,
    )

```

9.8 Скриншот запуска программы

```
python3 physics-2.1/main.py
Physics task 2.1: capacitor battery

Input data:
  C1 = C2 = C3 = C5 = 1.0 uF
  C4 = 0.5 uF
  U0 = 200.0 V

--- Open switch ---
  C12 = C1 * C2 / (C1 + C2) = 0.500000 uF
  C34 = C3 * C4 / (C3 + C4) = 0.333333 uF
  C_initial = C12 + C34 = 0.833333 uF
  Q_initial = C_initial * U0 = 166.666667 uC
  E_initial = C_initial * U0^2 / 2 = 0.016667 J

--- Closed switch: node potentials ---
  3.0 * Va - 1.0 * Vb = 1.0
 -1.0 * Va + 2.5 * Vb = 1.0
  Va = 0.538462 V
  Vb = 0.615385 V
  Q_left = 0.846154 uC at 1 V
  C_final = 0.846154 uF

--- Charge conservation after disconnecting the source ---
  Q_final = Q_initial = 166.666667 uC
  U_final = Q_initial / C_final = 196.969697 V

--- Energy ---
  E_final = Q^2 / (2 * C_final) = 0.016414 J
  Delta E = E_final - E_initial = -0.000253 J

Energy decreases by approximately 0.253 mJ.
```

Рис. 3: Результат запуска программы для физической задачи 2.1

9.9 Вывод по физической задаче

При замыкании ключа эквивалентная ёмкость батареи увеличивается:

$$C_{\text{нач}} \approx 0.8333 \text{ мкФ}, \quad C_{\text{кон}} \approx 0.8462 \text{ мкФ}.$$

Так как заряд сохраняется, напряжение на батарее падает с 200 В до ≈ 196.97 В. Энергия электрического поля уменьшается на ≈ 0.253 мДж. Потерянная энергия рассеивается в переходных процессах при замыкании ключа.

10 Вывод

В ходе лабораторной работы была решена система линейных алгебраических уравнений методом простой итерации. Проверка диагонального преобладания и норма матрицы итераций подтвердили сходимость метода. Полученное приближенное решение совпадает с контрольным ответом после округления:

$$X \approx (2.00; 1.00; 1.00).$$

Требуемая точность $\varepsilon = 0.01$ достигнута.